

Modulo 4

Python INSTITUTE

« 4.3.1.6 LABORATORIO: Un año bisiesto: escribiendo tus propias funciones »

MODULE (30%) SECTION (55%)

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la instrucción `return`.
- Probar las funciones.

Escenario

Tu tarea es escribir y probar una función que toma un argumento (un año) y devuelve `True` si el año es un *año bisiesto*, o `False` si no lo es.

Parte del esqueleto de la función ya está en el editor.

Nota: también hemos preparado un breve código de prueba, que puedes utilizar para probar tu función.

El código utiliza dos listas: una con los datos de prueba y la otra con los resultados esperados. El código te dirá si alguno de tus resultados no es válido.

```
1- def is_year_leap(year):
2-     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
3-         return True
4-     else:
5-         return False
6-
7- test_data = [1900, 2000, 2016, 1987]
8- test_results = [False, True, True, False]
9- for i in range(len(test_data)):
10-     yr = test_data[i]
11-     print(yr, ">>", end="")
12-     result = is_year_leap(yr)
13-     if result == test_results[i]:
14-         print("OK")
15-     else:
16-         print("Fallido")
```

Console >...

```
1900 ->OK
2000 ->OK
2016 ->OK
1987 ->OK
```

Prev Next

20°C Mayorm. nublado

19:12 23/09/2026

« 4.3.1.7 LABORATORIO: ¿Cuántos días?: escribiendo y utilizando tus propias funciones »

MODULE (32%) SECTION (54%)

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones parametrizadas.
- Utilizar la instrucción `return`.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que toma dos argumentos (un año y un mes) y devuelve el número de días del mes/año dado (mientras que solo febrero es sensible al valor `year`, tu función debería ser universal).

La parte inicial de la función está lista. Ahora, haz que la función devuelva `None` si los argumentos no tienen sentido.

Por supuesto, puedes (y debes) utilizar la función previamente escrita y probada (LABORATORIO 4.1.3.6). Puede ser muy útil. Te recomendamos que utilices una lista con los meses. Puedes crearla dentro de la función; este truco acortará significativamente el código.

Hemos preparado un código de prueba. Amplíalo para incluir más casos de prueba.

```
1- def is_year_leap(year):
2-     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
3-         return True
4-     else:
5-         return False
6-
7- def days_in_month(year, month):
8-     if year < 1 or month < 1 or month > 12:
9-         return None
10-
11-     month_days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
12-
13-     if month == 2 and is_year_leap(year):
14-         return 29
15-
16-     return month_days[month - 1]
17-
18- test_years = [1900, 2000, 2016, 1987]
19- test_months = [2, 2, 1, 11]
20- test_results = [28, 29, 31, 30]
21- for i in range(len(test_years)):
22-     yr = test_years[i]
23-     mo = test_months[i]
24-     print(yr, mo, ">>", end="")
25-     result = days_in_month(yr, mo)
26-     if result == test_results[i]:
```

Console >...

```
1900 2 ->OK
2000 2 ->OK
2016 1 ->OK
1987 11 ->OK
```

Prev Next

20°C Mayorm. nublado

19:14 23/09/2026

Python INSTITUTE

4.3.1.8 LABORATORIO: Día del año: escribiendo y utilizando tus propias funciones

MODULE (33%)

SECTION (73%)

Requisitos Previos

LABORATORIO 4.1.3.6

LABORATORIO 4.1.3.7

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la sentencia `return`.
- Construir un conjunto de funciones de utilidad.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que toma tres argumentos (un año, un mes y un día del mes) y devuelve el día correspondiente del año, o devuelve `None` si cualquiera de los argumentos no es válido.

Debes utilizar las funciones previamente escritas y probadas. Agrega algunos casos de prueba al código. Esta prueba es solo el comienzo.

```
13- if month == 2 and is_year_leap(year):
14-     return 29
15-
16- return month_days[month - 1]
17-
18- def day_of_year(year, month, day):
19-     # Validar el mes
20-     days_this_month = days_in_month(year, month)
21-     if days_this_month is None:
22-         return None
23-
24-     # Validar el día
25-     if day < 1 or day > days_this_month:
26-         return None
27-
28-     # Sumar los días de los meses anteriores
29-     total_days = 0
30-     for m in range(1, month):
31-         total_days += days_in_month(year, m)
32-
33-     # Sumar los días del mes actual
34-     total_days += day
35-
36-     return total_days
37-
38- print(day_of_year(2000, 12, 31))
```

Console >_

366

20°C

Mayorm. nublado

Buscar

19:15

23/09/2026

J

JORGE

1225100379.jam@gmail.com

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

Tu cuenta en Chrome

Cerrar sesión en Chrome

Otros perfiles de Chrome

Amed Yas

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python INSTITUTE

4.3.1.9 LABORATORIO: Números primos: ¿Cómo encontrarlos?

MODULE (33%)

SECTION (82%)

La función:

- Se llama `is_prime`.
- Toma un argumento (el valor a verificar).
- Devuelve `True` si el argumento es un número primo, y `False` de lo contrario.

Sugerencia: intenta dividir el argumento por todos los valores posteriores (comenzando desde 2) y verifica el resto: si es cero, tu número no puede ser un número primo; analiza cuidadosamente cuándo deberías detener el proceso.

Si necesitas conocer la raíz cuadrada de cualquier valor, puedes utilizar el operador `**`. Recuerda: la raíz cuadrada de x es lo mismo que $x^{0.5}$.

Complementa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Datos de prueba

Salida esperada:

2 3 5 7 11 13 17 19

```
1- def is_prime(num):
2-     if num <= 1:
3-         return False
4-     for i in range(2, int(num ** 0.5) + 1):
5-         if num % i == 0:
6-             return False
7-     return True
8-
9- for i in range(1, 20):
10-     if is_prime(i + 1):
11-         print(i + 1, end=" ")
12- print()
```

Console >_

2 3 5 7 11 13 17 19

20°C

Mayorm. nublado

Buscar

19:16

23/09/2026

J

JORGE

1225100379.jam@gmail.com

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

Tu cuenta en Chrome

Cerrar sesión en Chrome

Otros perfiles de Chrome

Amed Yas

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Edube Interactive - PE1 - Exam

Pregunta a Gemini

edube.org/quiz/python-essentials-1-esp-edube/pe1-examen-de-seccion

Python INSTITUTE

« PE1 - Examen De Sección »

Progreso (100%)

Tu puntaje: 30/35

86%

Congratulations, you've passed the Summary Test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Summary Test86%

Volver a presentar la prueba

Revisar prueba

WE use cookies to improve our service. By continuing to use the site, you agree to our [privacy](#) and [cookie policies](#).

20°C
Mayorm. nublado

Buscar

19:28
23/09/2026

Edube Interactive - Felicidades

Pregunta a Gemini

edube.org/learn/python-essentials-1-esp-edube/felicidades-1

Python INSTITUTE

« Felicidades »

Congratulations!

¡Felicidades!

¡Has completado el curso!

You've completed the course!

Prev

20°C
Mayorm. nublado

Buscar

19:29
23/09/2026

Modulo 4

Funciones, tuplas, diccionarios, y procesamiento de datos.

Si un fragmento de código comienza a aparecer en más de una ocasión, considera la posibilidad de aislarlo en la forma de una función invocando la función desde el lugar en el que originalmente se encontraba. Puede suceder que el algoritmo que se desea implementar sea tan complejo que el código comienza a crecer de manera incontrolada y de repente ya no se puede navegar por él tan fácilmente. Una función es un bloque de código que realiza una tarea específica cuando la función es llamada. Las funciones son útiles para hacer que el código sea reutilizable, que esté mejor organizado y más legible. Las funciones contienen parámetros y pueden regresar valores. Las funciones integradas las cuales son parte importante de python. El potencial completo de una función se revela cuando puede ser equipada con una interfaz que es capaz de aceptar datos provenientes de la invocación. Dichos datos pueden modificar el comportamiento de la función, haciéndola más flexible y adaptable a condiciones cambiantes.

Modelo 4

Un parametro es una variable pero existen dos factores que hacen a un parametro diferentes. Los parametros existen dentro de las funciones en donde se han definido y el unico lugar donde un parametro puede ser definido es entre los parentesis despues del nombre de la funcion donde se encuentra la palabra reservada `def`.

La asignacion de un valor a un parametro dentro de una funcion se hace en el momento en que la funcion se manda llamar o se invoca. Los parametros solo existen dentro de las funciones, Los argumentos existen de fuera de las funciones y son los que pasan los valores a los parametros correspondientes.

Especificar uno o mas parametros en la definicion de la funcion es un requerimiento y se debe cumplir durante la invocacion de la misma. Se debe proveer el mismo numero de argumentos como haya parametros definidos. El no hacerlo provocara un error. Una secuencia es un tipo de datos el cual es capaz de almacenar mas de un valor examinados. Lo primero que distingue una tupla es su sintaxis.